

Forever — iOS Weight Tracking App

Requirements Documentation

Date: June 2026

Version: 1.0

Status: Phases 1–3 implemented; Phases 4–9 planned

Introduction

Forever is a demo iOS application for tracking weight changes over time across multiple family member profiles. A single user operates the app and records weight measurements on behalf of multiple people (e.g. family members), each with their own customisable kitchen home screen and avatar. The app runs entirely on-device with no backend dependency.

Technology Stack

Concern	Technology	Version
Framework	React Native + Expo	SDK 56
Language	TypeScript	~5.8
Navigation	Expo Router (file-based)	~56.2
State management	Zustand	^5.0
Local database	expo-sqlite + Drizzle ORM	~56.0 / ^0.45
Authentication	expo-secure-store + crypto-js	~56.0 / ^4.2
SVG graphics	react-native-svg	installed
Charts	victory-native + @shopify/react-native-skia	^41.x / 2.6.2
Bottom sheet	@gorhom/bottom-sheet	^5.2
Date picker	react-native-modal-datetime-picker	^18.0
Deployment	Expo Go (demo) → EAS Build + TestFlight (distribution)	—

R1 — Simple Deployment

Requirement:

"The app is for demonstration and a short timeframe must be met so keeping the deployment as simple as possible is required."

How it is achieved:

The project uses React Native with Expo SDK 56. Expo's managed workflow eliminates virtually all native build complexity: there is no Xcode project to configure, no provisioning profiles to manage, and no App Store submission required for demos.

Demo day deployment: Running `npx expo start` starts the Metro bundler and displays a QR code. Any iPhone with the free Expo Go app installed can open the application immediately by scanning the QR code — no Apple Developer account, no device registration, no trust dialog. The app loads over the local network in seconds.

Wider distribution: If a build needs to be shared beyond the immediate demo, `eas build --platform ios --profile preview` produces a signed IPA that can be distributed via TestFlight to invited testers. This requires a free Apple Developer account but no App Store review.

Configuration: `app.json` sets `"platforms": ["ios"]` and `"scheme": "forever"`, scoping the project to iOS only with no Android scaffolding generated. The `expo-router` plugin, `expo-sqlite` plugin, `expo-secure-store` plugin, and `@react-native-community/datetimestamppicker` plugin are all declared in `app.json` and resolved automatically by Expo's build system.

Status: Implemented Key file: `app.json`

R2 — App Login

Requirement:

"The app will allow users to login."

How it is achieved:

Authentication is local-only — no backend server, no cloud auth service (e.g. AWS Cognito), and no network connection is required. This is intentional for demo reliability and simplicity.

First launch — Create Account: When the app starts, the root layout (`app/_layout.tsx`) calls `hasAccount()` from the auth service, which checks whether a credentials entry exists in the iOS Keychain via `expo-secure-store`. If no entry exists, the user is routed to `app/(auth)/create-account.tsx`, which collects a display name, password, and password confirmation.

On submit, `createAccount(displayName, password)` in `src/services/auth.service.ts`:

1. Generates a UUID to serve as the user ID
2. Generates a second UUID as a random salt
3. Computes `SHA-256(password + salt)` using `crypto-js`
4. Stores `{ userId, displayName, passwordHash, salt }` as encrypted JSON in the iOS Keychain under the key `"forever_credentials"` via `expo-secure-store`

Subsequent launches — Login: The app routes to `app/(auth)/login.tsx`, which presents a single password field. On submit, `login(password)` reads the stored credentials, re-computes `SHA-256(password + storedSalt)`, and compares to the stored hash. On match, it returns the stored `userId` and `displayName`; on mismatch, it returns `null` and an error message is shown.

Auth state: A Zustand store (`src/store/auth.store.ts`) holds `{ userId, displayName }`. The root layout (`app/_layout.tsx`) watches `userId` via a `useEffect` hook. When it becomes non-null after login, `initActiveProfile(userId)` is called and the router replaces to `//(tabs)`. When it is null (on logout), the router replaces to `//(auth)/login`.

Security note: The password is never stored in plaintext or reversibly encoded. The SHA-256 hash with UUID salt meets the security requirements for a local demo app. `expo-secure-store` encrypts the Keychain entry using iOS Data Protection, making it inaccessible to other apps and protected when the device is locked.

Status: Implemented Key files: `src/services/auth.service.ts`, `src/store/auth.store.ts`, `app/(auth)/login.tsx`, `app/(auth)/create-account.tsx`, `app/_layout.tsx`

R3 — Create Profiles

Requirement:

"The app will allow users to create 'profiles' in the app for different people. This will allow the user to track the weight on behalf of others. The idea being one person in the family can track the weight of other family members in one app."

How it is achieved:

Profiles are stored in the `profiles` table in the local SQLite database, managed by Drizzle ORM (`src/db/schema.ts`). Each profile has a `userId` foreign key linking it to the single app user, enabling multiple profiles per user.

Profile list screen (`app/(tabs)/profiles.tsx`): The Profiles tab renders a `FlatList` of all profiles belonging to the current user, fetched via `getProfiles(userId)` from `src/services/profile.service.ts` (Drizzle query ordered by name). The currently active profile is highlighted with a blue border. A floating action button (FAB) in the bottom right navigates to the Create Profile screen.

Creating a profile (`app/(tabs)/create-profile.tsx`): The form — implemented as a shared `ProfileForm` component (`src/components/shared/ProfileForm.tsx`) — collects:

- **Name** (free text, required)
- **Weight unit** (kg / lbs / stones — segmented control)
- **Avatar gender** (Male / Female toggle)
- **Avatar body width** (Slim / Regular / Athletic / Larger)
- **Avatar skin tone** (four colour swatches)
- **Avatar hair style** (Short / Long / Curly / Shaved or Bun)
- Kitchen variant fields (table, bowl, plantpot, wall — stored as integers 0–3, configured via the kitchen customisation screen in Phase 5)

On save, `createProfile(userId, values)` inserts a new row with a UUID primary key and the current timestamp.

Editing and deleting (`app/(tabs)/edit-profile.tsx`): Tapping any profile row navigates to the Edit Profile screen, which pre-populates the same `ProfileForm` with the existing profile's values. Saving calls `updateProfile(id, values)`. A trash icon on the profile list row triggers an `Alert.alert` confirmation; confirming calls `deleteProfile(id)`, which removes the profile row. Because the `measurements` table has `ON DELETE CASCADE` on `profileId`, all of that profile's measurements are automatically removed.

Status: Implemented Key files: `app/(tabs)/profiles.tsx`, `app/(tabs)/create-profile.tsx`, `app/(tabs)/edit-profile.tsx`, `src/services/profile.service.ts`, `src/components/shared/ProfileForm.tsx`

R4 — App Visuals / Kitchen Home Screen

Requirement:

"The visuals of the app are important and the background of the home screen should be customisable. The background should show a simple kitchen scene. This includes a table, a bowl on the table, a plant in a plantpot at the side of the table, and a wall at the back. For each object there will be four options the user can choose — 4 table colours, 4 bowl colours, 4 plantpot shapes and 4 wall colours. The user has complete choice to choose any combination they wish for a given profile. The Home Screen must show the choices chosen for the profile in the kitchen setup."

How it is achieved (design):

The kitchen scene is drawn programmatically using `react-native-svg` — no external image files are required. Each kitchen element is a self-contained React component built from SVG primitives (`Rect` , `Ellipse` , `Path` , `G`).

Scene composition: `KitchenScene` is a stateless component that accepts a `KitchenConfig` object containing four integer variant indices (0–3) and renders a `<Svg viewBox="0 0 400 300">` with layered elements:

- **Layer 0 — Wall:** Full background rectangle; 4 colour variants
- **Layer 1 — Table:** Lower half of the scene; 4 colour variants
- **Layer 2 — Bowl:** Centred on the table surface; 4 colour variants
- **Layer 3 — Plantpot:** Right side of scene; 4 distinct shape variants (different SVG path strings)
- **Layer 4 — Avatar:** Standing figure (see R11); added in Phase 6

The aspect ratio is fixed at 4:3 and the component scales to any screen width. Colour variants change the SVG `fill` attribute; shape variants (plantpot) swap between different SVG path strings.

Variant storage: All four variant selections are stored as integer columns (`tableVariant` , `bowlVariant` , `plantpotVariant` , `wallVariant`) on the `profiles` table, so each profile maintains its own independent kitchen configuration.

Kitchen customisation screen: `CustomiseKitchenScreen` shows four horizontal selection rows, one per element, each displaying four thumbnail previews. Tapping a thumbnail calls `updateProfile()` , which updates the profile row and syncs the Zustand active profile store. Because the Home screen subscribes to the active profile store, the kitchen scene re-renders immediately without navigation.

Status: Planned — Phase 5 Key files to create: `src/components/kitchen/KitchenScene.tsx` , `src/components/kitchen/KitchenTable.tsx` , `src/components/kitchen/KitchenBowl.tsx` , `src/components/kitchen/KitchenPlantpot.tsx` , `src/components/kitchen/KitchenWall.tsx` , `src/screens/CustomiseKitchenScreen.tsx` , `app/(tabs)/index.tsx` (replace stub)

R5 — User Profile Switching

Requirement:

"The user must be able to switch between profiles. The app will reflect which profile is active."

How it is achieved:

The active profile is held in a global Zustand store (`src/store/activeProfile.store.ts`) with fields `{ profile, setProfile, clearProfile }` . Because all screens subscribe to this store, switching profiles causes every screen to re-render with the new profile's data simultaneously — no navigation is required.

ProfileSwitcher component (`src/components/shared/ProfileSwitcher.tsx`): A `Pressable` mounted in the tab navigator's header (`app/(tabs)/_layout.tsx`) so it is visible from every tab. It displays the active profile's name with a chevron icon. Tapping it opens a `@gorhom/bottom-sheet` bottom sheet listing all profiles fetched from the database. The active profile has a checkmark. Tapping any row calls `setProfile(profile)` from the active profile store, immediately updating all subscribed screens.

Persistence: The active profile ID is persisted to `AsyncStorage` inside `setProfile()` so the same profile is restored on the next app launch. On login, `initActiveProfile(userId)` reads the saved ID from `AsyncStorage` and selects that profile (or falls back to the first profile alphabetically if the saved ID no longer exists).

Status: Implemented Key files: `src/store/activeProfile.store.ts` , `src/components/shared/ProfileSwitcher.tsx` , `app/(tabs)/_layout.tsx` , `src/services/profile.service.ts`

R6 — Tracking Measurements

Requirement:

"The user should be able to input a weight measurement for a profile. This measurement must be saved so that weight changes over time can be tracked. Historical measurements need to be accepted as well as live measurements."

Additional agreed requirement: Weight input supports three units — kilograms (kg), pounds (lbs), and stones/pounds (st). The unit preference is set per profile. All measurements are stored internally in kg regardless of the input unit.

How it is achieved (design):

`AddMeasurementScreen` is a modal that reads the active profile's `weightUnit` and renders the appropriate input field(s):

- **kg or lbs:** A single `TextInput` with `keyboardType="decimal-pad"` and a unit suffix label
- **Stones:** Two `TextInput` fields side by side — `[11] st [4] lbs` — both with `keyboardType="number-pad"`

A date row displays the selected date formatted as `dd MMM yyyy`. Tapping it opens a `DateTimePickerModal` from `react-native-modal-datetime-picker` with a minimum date of 1 Jan 1900 and a maximum of today. This allows both historical measurements (e.g. a weight recorded on a birthday years ago) and same-day entries.

Optional fields include a free-text notes input and a milestone toggle (see R8).

On save, the weight input is converted to kg using `toKg(value, unit, lbsPart?)` from `src/utils/weight.ts` before being inserted into the `measurements` table. The `measurements` table stores `weightKg` (always kg), `measuredAt` (Unix millisecond timestamp), `profileId`, and optional milestone fields.

Weight unit conversion (`src/utils/weight.ts` , already implemented):

- `toKg(primary, unit, lbsPart?)` — converts any unit to kg for storage
- `fromKg(kg, unit) → { primary, secondary? }` — stones returns `{ primary: stones, secondary: lbs }`
- `formatWeight(kg, unit) → "72.5 kg" / "159.8 lbs" / "11 st 4 lbs"`

Long-pressing any measurement in the timeline opens the same modal pre-populated with that measurement's values, enabling editing.

The `measurements` table has a composite index on `(profileId, measuredAt)` for fast ordered timeline queries.

Status: Planned — Phase 4 Key files to create: `src/screens/AddMeasurementScreen.tsx` ,
`src/services/measurement.service.ts` Already implemented: `src/utils/weight.ts`

R7 — Displaying Measurements

Requirement:

"Two different viewing options should be available to the user. The first being a scrollable timeline where all measurements are displayed in order of date the measurement was taken. The second measurement view should be a plot view — a line graph with weight on the y-axis and date/time on the x-axis."

How it is achieved (design):

The Measurements tab (`app/(tabs)/measurements.tsx`) hosts two views accessible via a top tab bar: **Timeline** and **Graph**.

Timeline view: A `FlatList` driven by a Drizzle query ordering measurements newest-first by `measuredAt`. Each

row is a `MeasurementTile` component showing:

- Date formatted as `dd MMM yyyy`
- Weight formatted in the profile's unit using `formatWeight(weightKg, profile.weightUnit)`
- A `MilestoneBadge` if the measurement is a milestone (see R8)

A segmented filter control at the top switches between "All" measurements and "Milestones only", adding a `WHERE isMilestone = 1` clause when active.

Graph view: Uses `victory-native` (`CartesianChart` with `Line` and `Scatter`) to plot weight over time. Data points are mapped from Drizzle query results (ordered by `measuredAt ASC`) to `{ x: timestamp, y: weight }` objects.

Y-axis behaviour by unit:

- **kg / lbs:** Y values are displayed in the profile's native unit; axis label shows "kg" or "lbs"
- **Stones:** Y values are stored and displayed as lbs on the graph axis (decimal stones values like "11.28 st" are unreadable); the tooltip converts back to `"11 st 4 lbs"` using `formatWeight()`

Milestone data points render as larger amber-coloured scatter markers, distinguishable from regular data points on the line. Tapping a data point shows a tooltip with the exact date and formatted weight.

Status: Planned — Phase 4 (Timeline) and Phase 7 (Graph) Key files to create: `src/screens/TimelineScreen.tsx` , `src/screens/GraphScreen.tsx` , `app/(tabs)/measurements.tsx` (replace stub), `src/components/measurement/MeasurementTile.tsx`

R8 — Milestones

Requirement:

"The user should be able to create milestone measurements for the profiles. The milestone measurements would be special measurements such as a weight measurement taken on the profile's birthday. The user should be able to create milestones. The user should have the option to view only the milestone measurements."

How it is achieved (design):

Milestones are implemented as a property of an existing measurement rather than a separate entity. The `measurements` table includes two dedicated columns:

- `isMilestone` — integer (0 or 1), default 0
- `milestoneName` — nullable text

This approach avoids duplicating measurement data and keeps the data model simple.

Creating milestones: The `AddMeasurementScreen` includes a "Mark as milestone" toggle switch. When enabled, a milestone name text input is revealed. Milestones can be set at the time of creating a new measurement or retroactively by editing an existing one via the long-press edit modal.

Displaying milestones: In the Timeline view, milestone measurements display a `MilestoneBadge` component — a small amber chip with a star icon and the milestone name — rendered inline within the `MeasurementTile` row.

Filtering milestones: The segmented filter control at the top of the Timeline view has an "All" option and a "Milestones" option. Selecting "Milestones" adds `WHERE isMilestone = 1` to the Drizzle query, showing only milestone measurements.

Milestones on the graph: Milestone data points are rendered as a second `Scatter` layer on the graph with a distinct larger amber marker, so they are immediately identifiable on the weight-over-time line.

Status: Planned — Part of Phase 4 (Timeline) and Phase 7 (Graph)

R9 — Collaborative View

Requirement:

"Where the user can see all the people/profiles on one page next to each other with the more recent milestone for that profile displayed. E.g. Parent 1 is 75kg and their most recent milestone was their 40th birthday, Parent 2 is 60kg and most recent milestone was a new job."

How it is achieved (design):

The Collaborative tab (`app/(tabs)/collaborative.tsx`) shows a horizontally scrollable row of `ProfileSummaryCard` components — one card per profile, always showing all profiles regardless of which is currently active.

Each card displays:

- Profile name
- A miniaturised kitchen scene thumbnail (the full `KitchenScene` component scaled down via `transform: [{ scale: 0.4 }]` inside an `overflow: 'hidden'` container — so each card shows that profile's unique kitchen configuration and avatar)
- The latest weight measurement formatted in that profile's weight unit (`formatWeight(weightKg, profile.weightUnit)`)
- The most recent milestone label and date (or "No milestone yet" if none exists)

Cards with no measurements show "No measurements yet" in place of the weight value.

Data loading: A custom hook `useCollaborativeData(userId)` runs all profile and measurement queries in parallel using `Promise.all`. For each profile it fetches: (1) the latest measurement ordered by `measuredAt DESC LIMIT 1`, and (2) the latest milestone measurement (`WHERE isMilestone = 1 ORDER BY measuredAt DESC LIMIT 1`). The hook re-fetches whenever the Collaborative tab comes into focus via `useFocusEffect`.

This screen has no dependency on the active profile store — it always reads and displays all profiles.

Status: Planned — Phase 8 Key files to create: `app/(tabs)/collaborative.tsx` (replace stub), `src/hooks/useCollaborativeData.ts`, `src/components/ProfileSummaryCard.tsx`

R10 — iOS Only

Requirement:

"The app will be only iOS to start with."

How it is achieved:

The project is scoped to iOS at every level:

- `app.json` sets `"platforms": ["ios"]`, preventing Android builds from being initiated
- `app.json` sets `"ios": { "bundleIdentifier": "com.demo.forever", "supportsTablet": false }`, configuring the app as an iPhone-only iOS application
- All third-party packages chosen have iOS support. `expo-secure-store` maps directly to the iOS Keychain (`SecItem` API). `expo-sqlite` uses the iOS-bundled SQLite. `@shopify/react-native-skia` (required by victory-native) uses Apple's Metal GPU API on iOS
- For the demo, Expo Go itself provides the iOS native runtime — no custom native build, no provisioning, and no Xcode installation is needed on the demo device

No Android-specific code, configuration, or native module wiring is present in the project.

Status: Implemented Key file: `app.json`

R11 — Customisable Profile Avatar (standing in kitchen scene)

Requirement:

"The user should be able to create an avatar for each profile. The user should be able to choose from 4 different widths of avatar, 4 different skin colours, 4 different hair styles and choose a male or female. The avatar should appear standing in the kitchen scene."

How it is achieved:

Data model (implemented): Avatar customisation options are stored as four columns on the `profiles` table in `src/db/schema.ts` :

- `avatarGender` — text: 'male' or 'female'
- `avatarWidth` — integer 0–3 (Slim / Regular / Athletic / Larger)
- `avatarSkinTone` — integer 0–3 (four skin tone colours: `#FDDBB4` , `#D4956A` , `#A0522D` , `#4A2912`)
- `avatarHairStyle` — integer 0–3 (Short / Long / Curly / Shaved or Bun)

This gives $2 \times 4 \times 4 \times 4 = 128$ **unique avatar combinations** without any image assets.

Profile form controls (implemented): The `ProfileForm` component (`src/components/shared/ProfileForm.tsx`) already includes all four avatar selection controls:

- A two-button gender toggle (Male / Female)
- A four-button body width row (Slim / Regular / Athletic / Larger)
- A four-circle skin tone swatch row (colour-coded)
- A four-button hair style row (Short / Long / Curly / Shaved/Bun)

These controls are displayed on both the Create Profile and Edit Profile screens. All selections are persisted to the database when the profile is saved.

Avatar rendering (planned — Phase 6): `KitchenAvatar` will be a programmatic SVG component using `react-native-svg` primitives — no external image files. It contains three sub-layers:

- `AvatarBody` — torso and legs, shape driven by width variant and gender
- `AvatarFace` — head circle and neck, fill colour driven by skin tone index
- `AvatarHair` — hair shape, SVG path string driven by hair style and gender combination

The avatar will be positioned in `KitchenScene` as Layer 4, standing left-of-centre with feet at the table surface line ($y \approx 178$ in the 400×300 SVG coordinate space), giving the appearance of a person standing at the kitchen table.

Status: Data model and form controls implemented. Avatar rendering component not yet built (Phase 6)

Key files: `src/db/schema.ts` ✓, `src/components/shared/ProfileForm.tsx` ✓ Key files to create:

`src/components/kitchen/KitchenAvatar.tsx`

Data Model Reference

Three SQLite tables managed by Drizzle ORM, defined in `src/db/schema.ts` :

`users`

Column	Type	Notes
<code>id</code>	text PRIMARY KEY	UUID
<code>displayName</code>	text NOT NULL	Set at account creation
<code>createdAt</code>	integer NOT NULL	Unix milliseconds

profiles

Column	Type	Notes
<code>id</code>	text PRIMARY KEY	UUID
<code>userId</code>	text NOT NULL	Foreign key → <code>users.id</code>
<code>name</code>	text NOT NULL	Profile display name
<code>weightUnit</code>	text	'kg' / 'lbs' / 'st'
<code>avatarGender</code>	text	'male' / 'female'
<code>avatarWidth</code>	integer	0–3 (Slim → Larger)
<code>avatarSkinTone</code>	integer	0–3
<code>avatarHairStyle</code>	integer	0–3
<code>tableVariant</code>	integer	0–3
<code>bowlVariant</code>	integer	0–3
<code>plantpotVariant</code>	integer	0–3
<code>wallVariant</code>	integer	0–3
<code>createdAt</code>	integer NOT NULL	Unix milliseconds

measurements

Column	Type	Notes
<code>id</code>	text PRIMARY KEY	UUID
<code>profileId</code>	text NOT NULL	Foreign key → <code>profiles.id</code> (CASCADE DELETE)
<code>weightKg</code>	real NOT NULL	Always stored in kg regardless of input unit
<code>measuredAt</code>	integer NOT NULL	Unix milliseconds
<code>isMilestone</code>	integer	0 or 1
<code>milestoneName</code>	text	Nullable
<code>notes</code>	text	Nullable
<code>createdAt</code>	integer NOT NULL	Unix milliseconds

Indexes:

- `(profileId, measuredAt)` — fast ordered timeline queries
- `(profileId, isMilestone)` — fast milestone filter queries

Key design decisions:

- Weights are always stored in kg. The `weightUnit` on the profile controls input presentation and display formatting only. All conversion is handled in `src/utlis/weight.ts`.

- Profile deletion cascades to all measurements via SQLite `ON DELETE CASCADE` , ensuring no orphaned measurement rows.

Remaining Work

The following phases have not yet been implemented at the time of this document:

Phase	Description
Phase 4	Measurement service, Add/Edit Measurement modal, Timeline view, MeasurementTile, MilestoneBadge
Phase 5	Kitchen scene SVG components (Wall, Table, Bowl, Plantpot), Home screen, Kitchen customisation screen
Phase 6	KitchenAvatar SVG component (Body, Face, Hair), integration into KitchenScene
Phase 7	Graph screen (Victory Native CartesianChart with Line, Scatter, tooltip)
Phase 8	Collaborative screen, useCollaborativeData hook, ProfileSummaryCard
Phase 9	Polish, full verification checklist, Expo Go QR demo test